

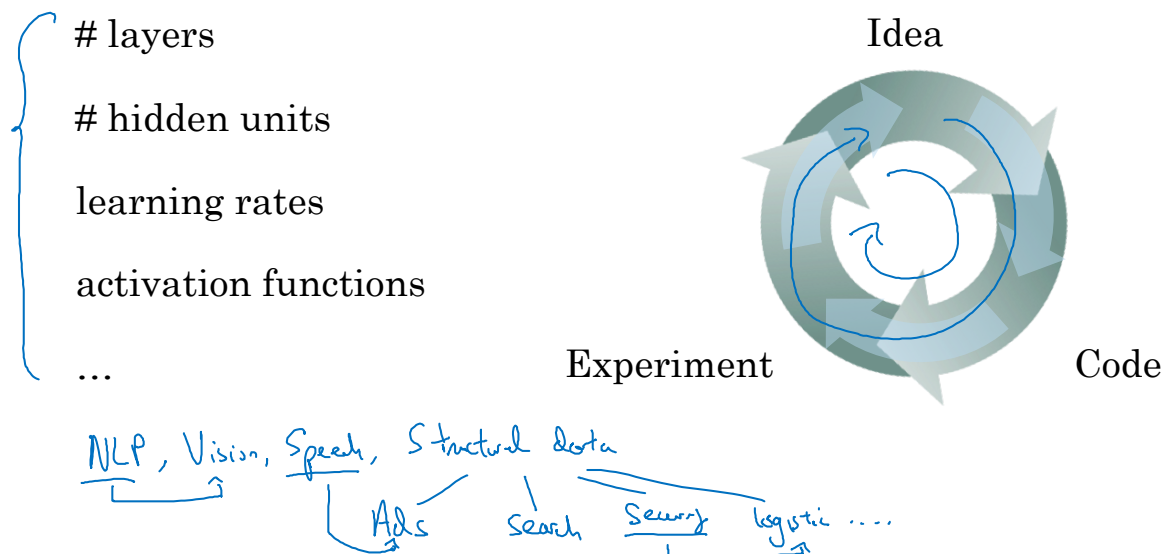


deeplearning.ai

Setting up your ML application

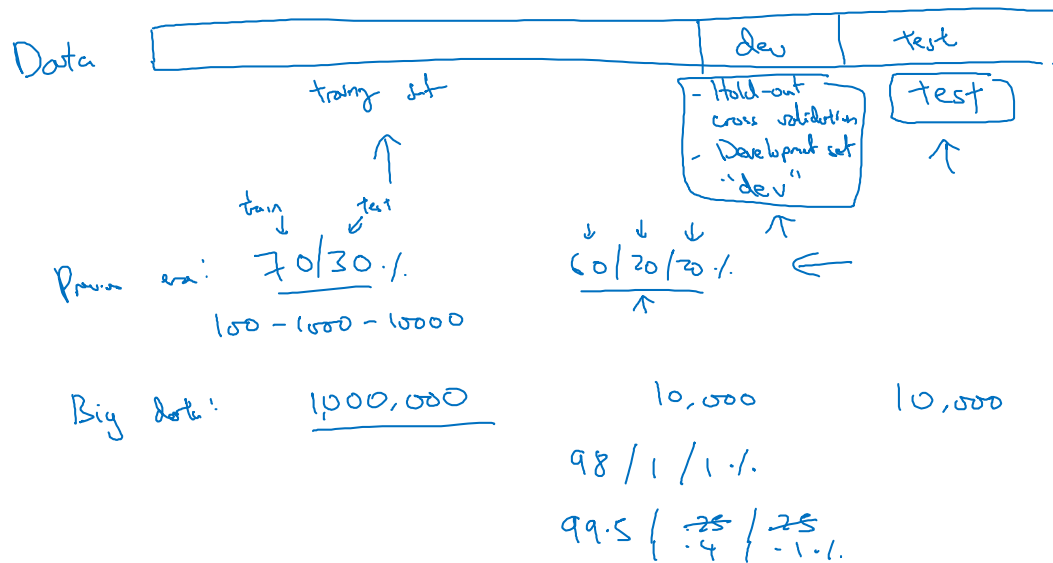
Train/dev/test sets

Applied ML is a highly iterative process



Andrew Ng

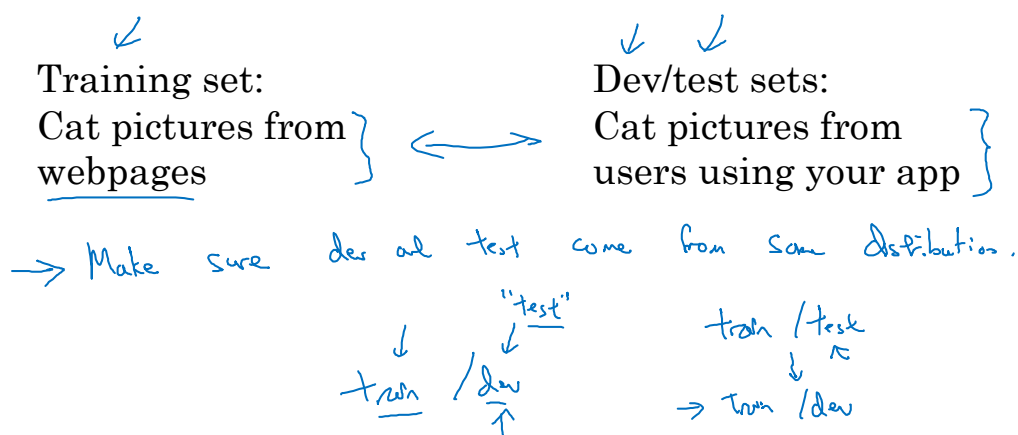
Train/dev/test sets



Andrew Ng

Mismatched train/test distribution

Certs



Not having a test set might be okay. (Only dev set.)

Andrew Ng

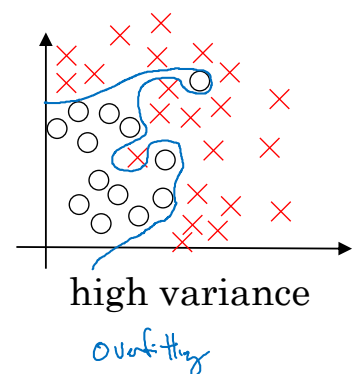
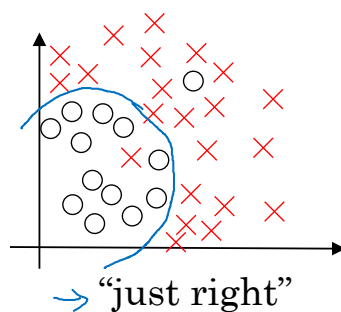
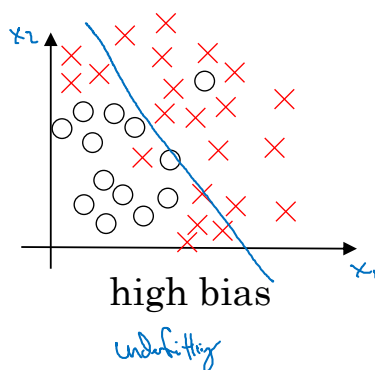


deeplearning.ai

Setting up your ML application

Bias/Variance

Bias and Variance



Bias and Variance

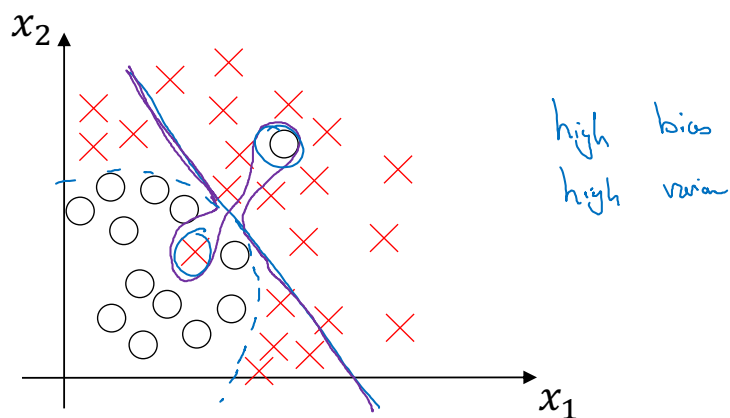
Cat classification



Train set error:	1%	15% ↙	15%	0.5%
Dev set error:	11%	16% ↙	30%	1%
	high variance ↑	high bias ↑↑	high bias & high variance	low bias low variance ↑
Human: 0%				
Optimal (Bayes) error: 15%				
			Blurry images	

Andrew Ng

High bias and high variance



Andrew Ng

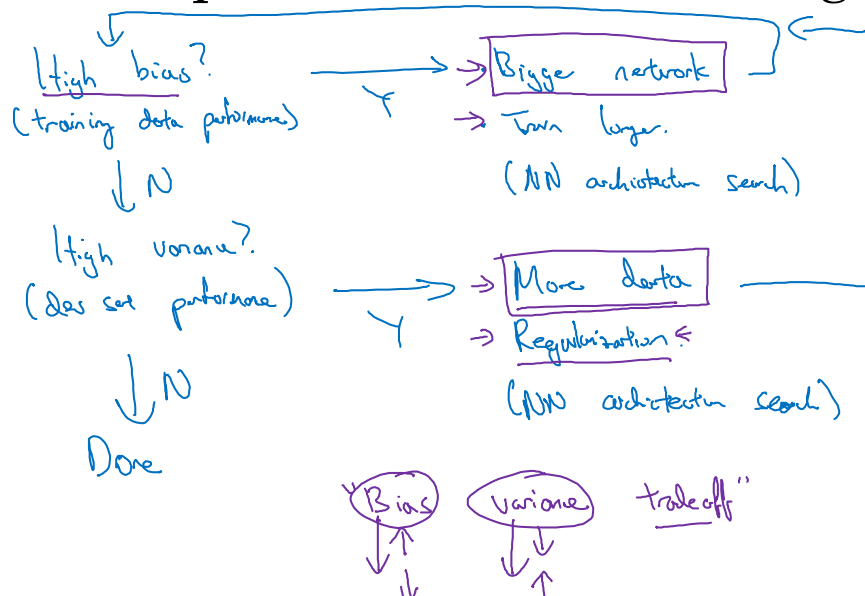


deeplearning.ai

Setting up your ML application

Basic “recipe” for machine learning

Basic recipe for machine learning





deeplearning.ai

Regularizing your neural network

Regularization

Logistic regression

$$\min_{w,b} J(w,b)$$

$$w \in \mathbb{R}^{n_x}, b \in \mathbb{R}$$

λ = regularization parameter
lambda lambda

$$J(w,b) = \underbrace{\frac{1}{m} \sum_{i=1}^m \ell(y^{(i)}, \hat{y}^{(i)})}_{\text{cost function}} + \frac{\lambda}{2m} \|w\|_2^2$$

~~$+\frac{\lambda}{2m} b^2$~~
omit

L_2 regularization $\|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w \leftarrow$

L_1 regularization $\frac{\lambda}{2m} \sum_{j=1}^{n_x} |w_j| = \frac{\lambda}{2m} \|w\|_1$

w will be sparse

Neural network

$$\rightarrow J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \underbrace{\frac{1}{m} \sum_{i=1}^m \ell(y^{(i)}, \hat{y}^{(i)})}_{\text{Frobenius norm}} + \underbrace{\frac{\lambda}{2m} \sum_{l=1}^L \|w^{[l]}\|_F^2}_{\| \cdot \|_2^2 \quad \| \cdot \|_F^2}$$

$$\|w^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (w_{ij}^{[l]})^2 \quad w^{[l]}: \begin{matrix} n^{[l]} & n^{[l-1]} \\ \uparrow & \uparrow \end{matrix}$$

$$dw^{[l]} = \left[\text{(from backprop)} + \frac{\lambda}{m} w^{[l]} \right] \quad \frac{\partial J}{\partial w^{[l]}} = dw^{[l]}$$

$$\rightarrow w^{[l]} := w^{[l]} - \alpha \left[\text{(from backprop)} + \frac{\lambda}{m} w^{[l]} \right]$$

"Weight decay"

$$\begin{aligned} w^{[l]} &:= w^{[l]} - \alpha \left[\text{(from backprop)} + \frac{\lambda}{m} w^{[l]} \right] \\ &= w^{[l]} - \frac{\alpha \lambda}{m} w^{[l]} - \alpha \text{(from backprop)} \\ &= \underbrace{\left(1 - \frac{\alpha \lambda}{m}\right)}_{< 1} \underline{w^{[l]}} - \alpha \text{(from backprop)} \end{aligned}$$

Andrew Ng

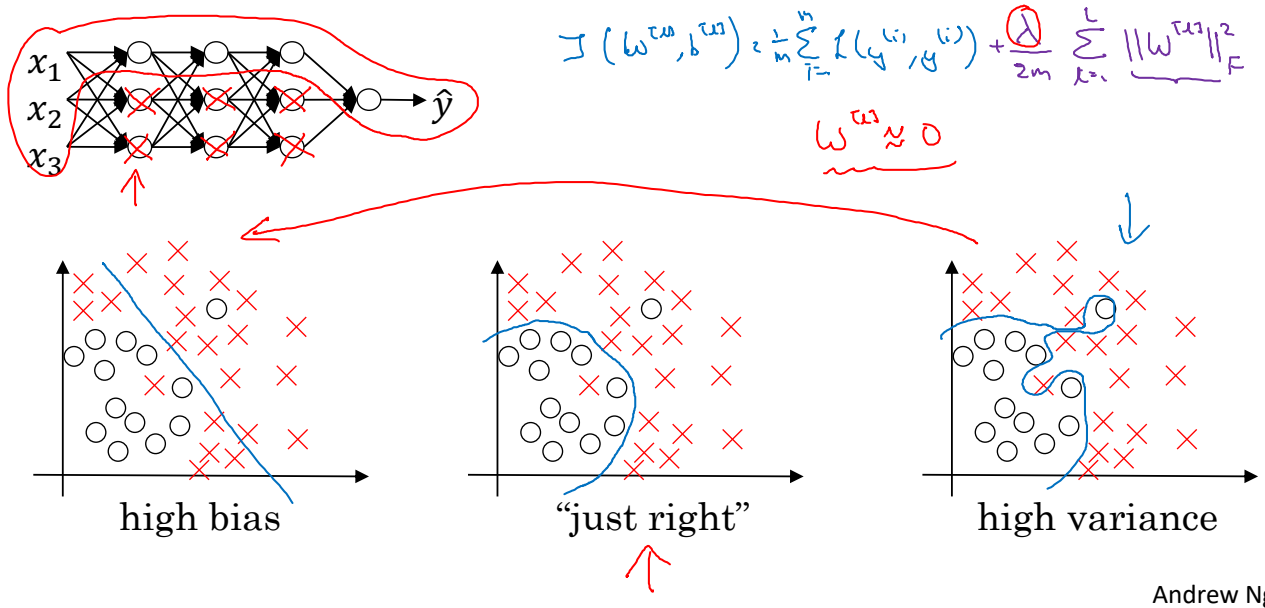


deeplearning.ai

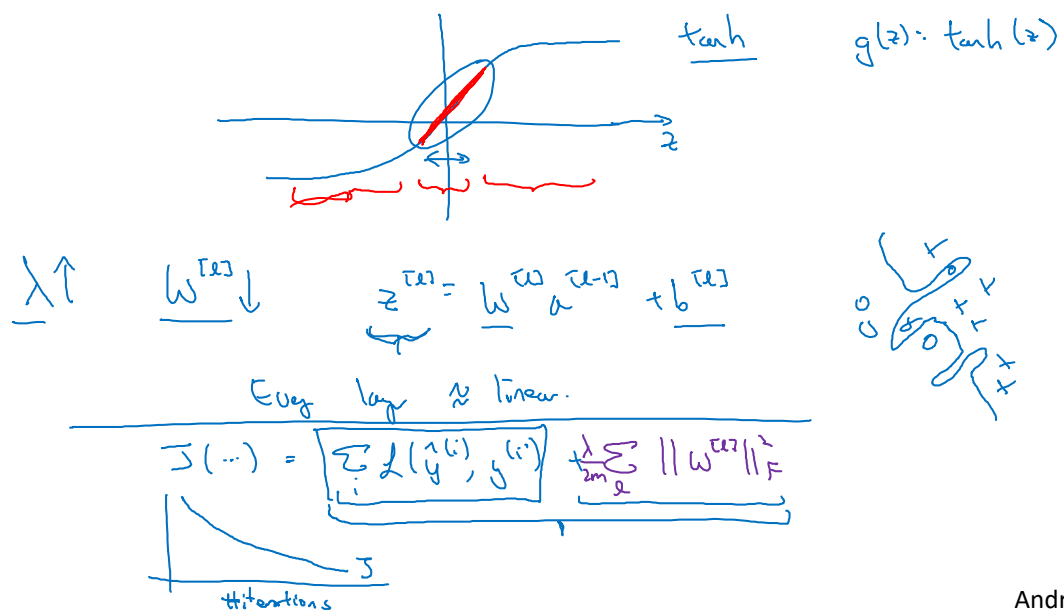
Regularizing your neural network

Why regularization reduces overfitting

How does regularization prevent overfitting?



How does regularization prevent overfitting?



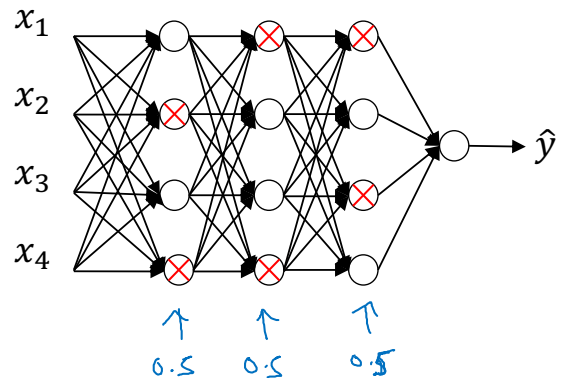
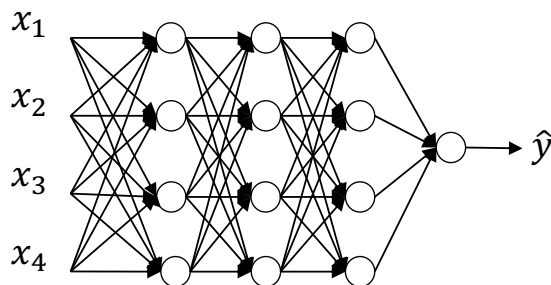


deeplearning.ai

Regularizing your neural network

Dropout regularization

Dropout regularization



Implementing dropout ("Inverted dropout")

Illustrate with layer $l=3$. $\text{keep-prob} = \frac{0.8}{x}$ 0.2

$\rightarrow d3 = \text{np.random.rand}(a3.\text{shape}[0], a3.\text{shape}[1]) < \text{keep-prob}$

$a3 = \text{np.multiply}(a3, d3)$ # $a3 \times d3$.

$\rightarrow a3 /= \text{keep-prob}$ ←

50 units. $\rightsquigarrow$ 10 units shut off

$z^{(4)} = w^{(4)} \cdot \underbrace{a^{(3)}}_{\substack{\uparrow \\ \text{reduced by } 20\% \\ /= 0.8}} + b^{(4)}$ Test

Andrew Ng

Making predictions at test time

$$a^{(0)} = X$$

No drop out.

$z^{(1)} = w^{(1)} a^{(0)} + b^{(1)}$

$a^{(1)} = g^{(1)}(z^{(1)})$

$z^{(2)} = w^{(2)} a^{(1)} + b^{(2)}$

$a^{(2)} = \dots$

↓

$\hat{y}$

$/= \text{keep-prob}$

Andrew Ng



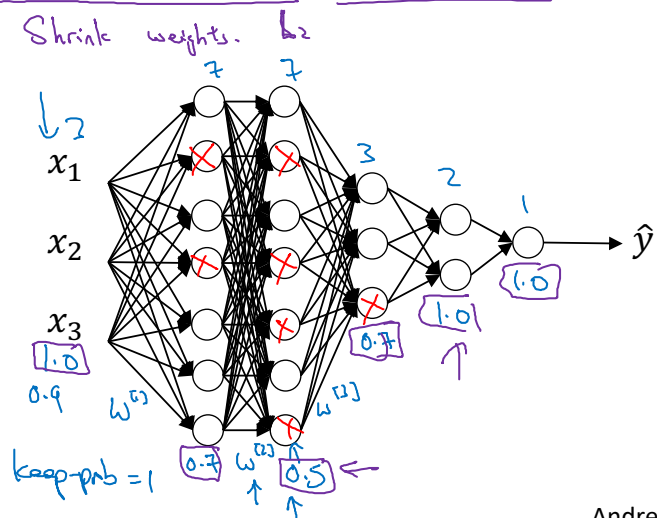
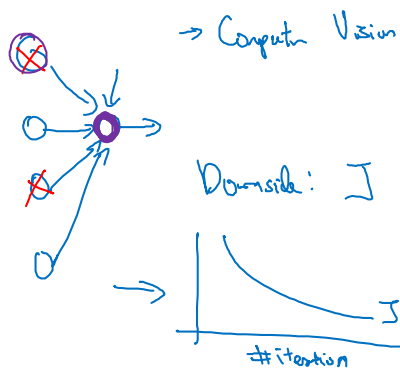
deeplearning.ai

Regularizing your neural network

Understanding dropout

Why does drop-out work?

Intuition: Can't rely on any one feature, so have to spread out weights. $\leadsto$ Shrink weights.



Andrew Ng

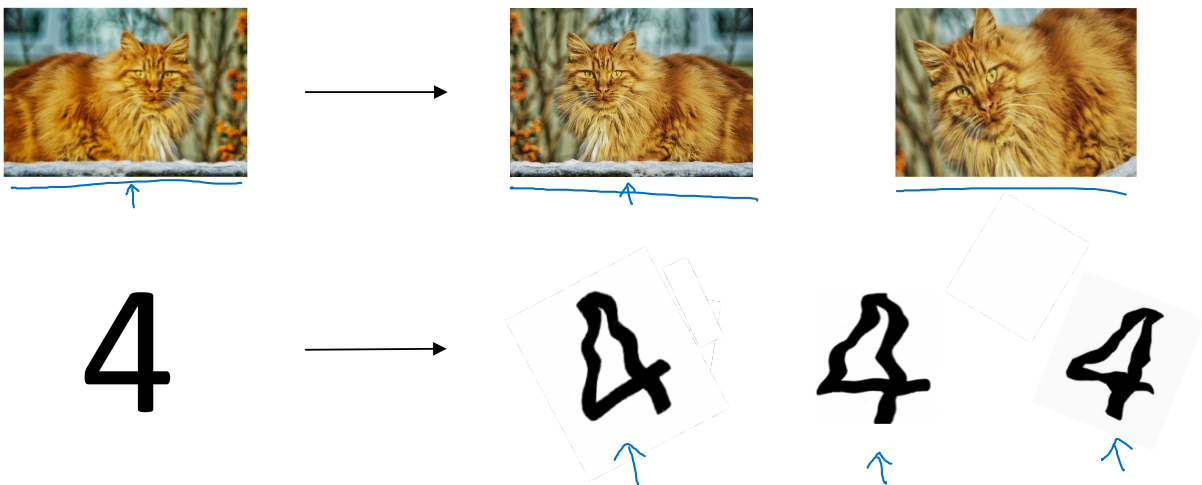


deeplearning.ai

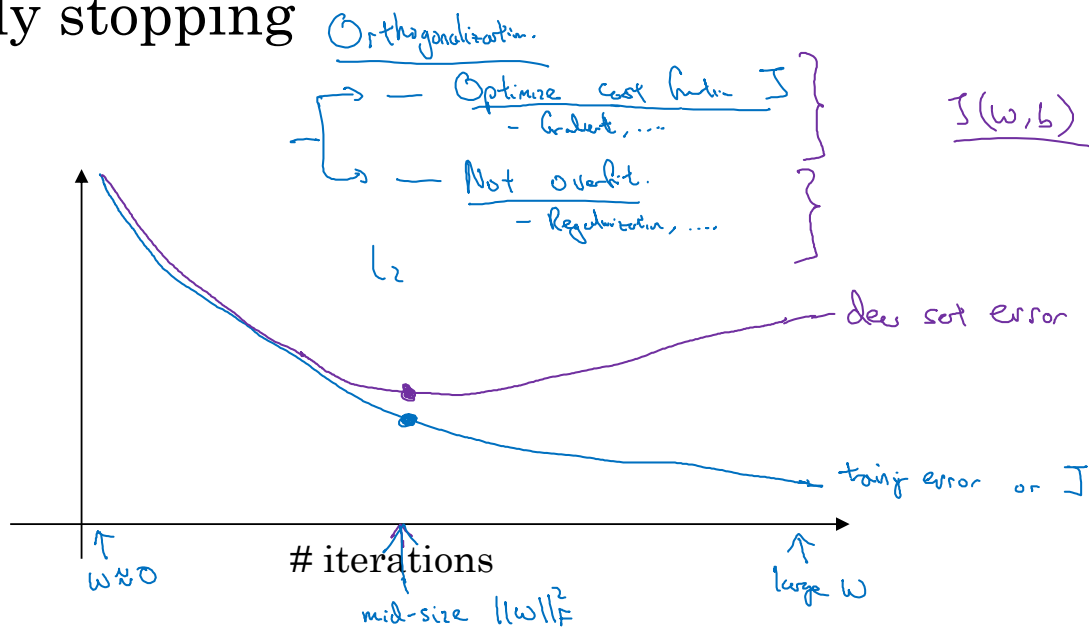
Regularizing your neural network

Other regularization methods

Data augmentation



Early stopping



Andrew Ng

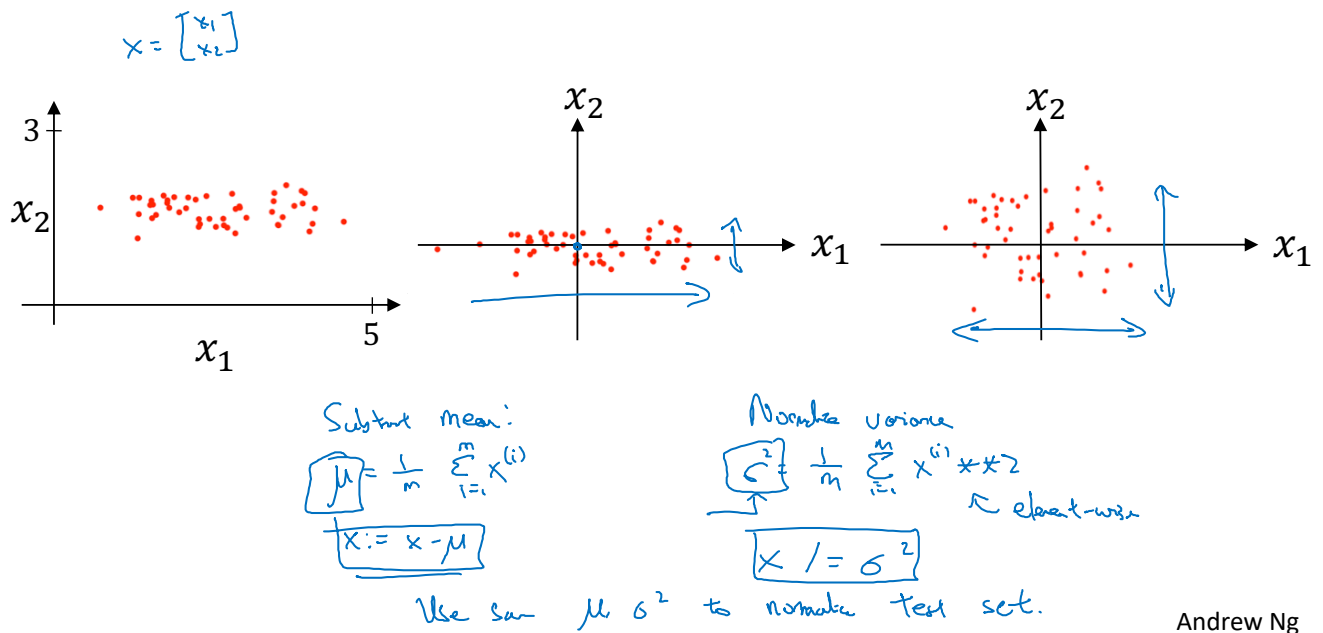


deeplearning.ai

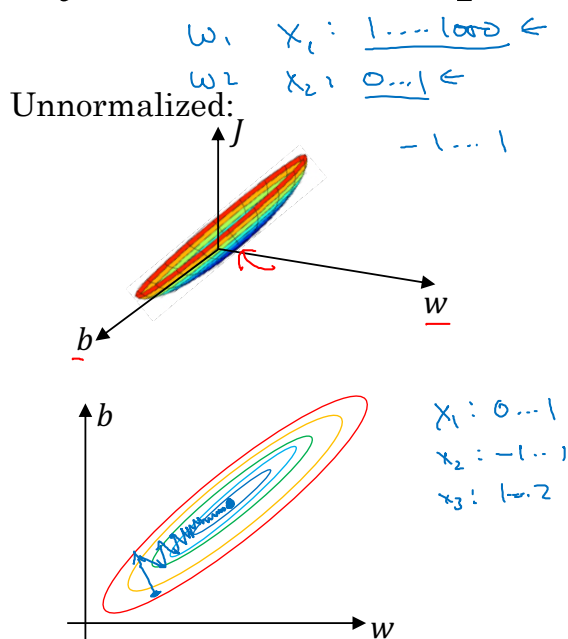
Setting up your optimization problem

Normalizing inputs

Normalizing training sets

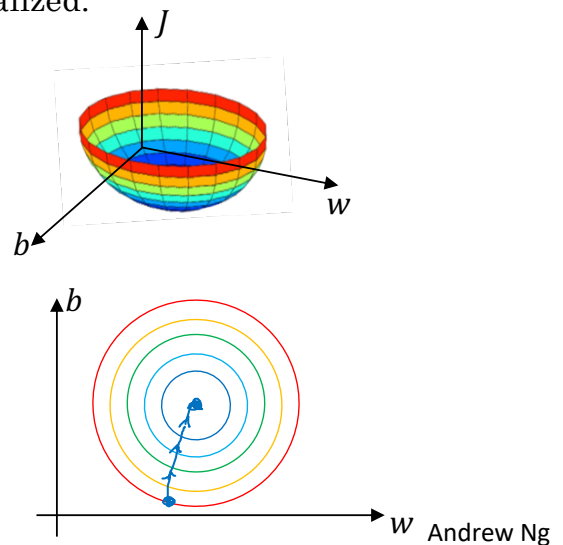


Why normalize inputs?



$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Normalized:



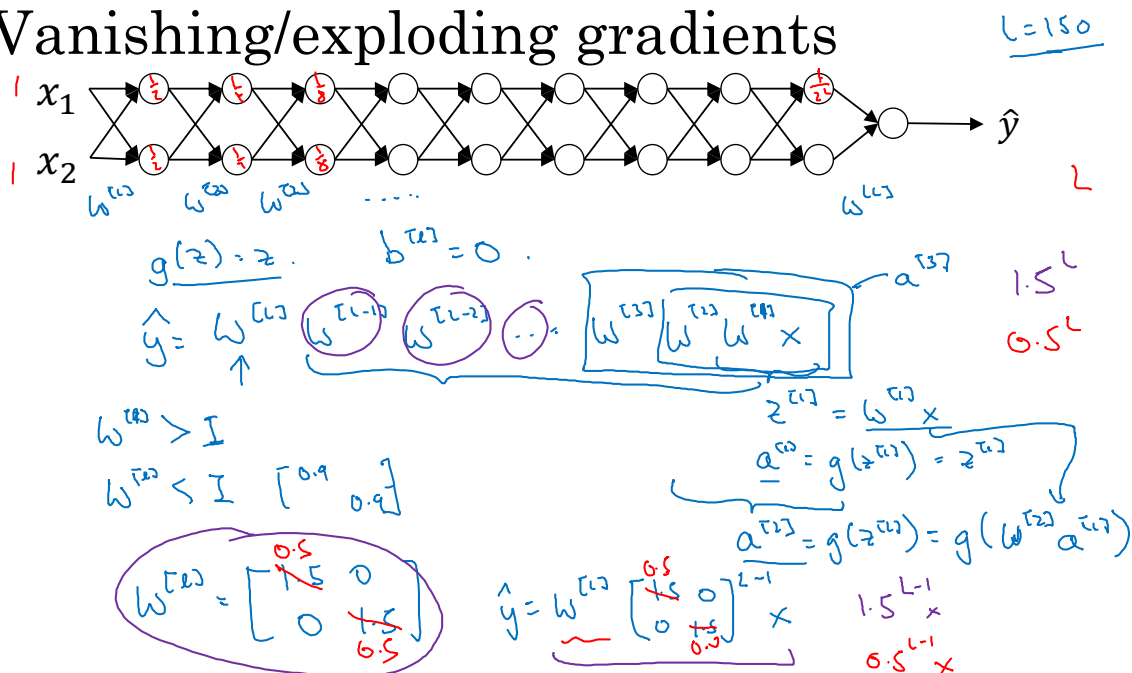


deeplearning.ai

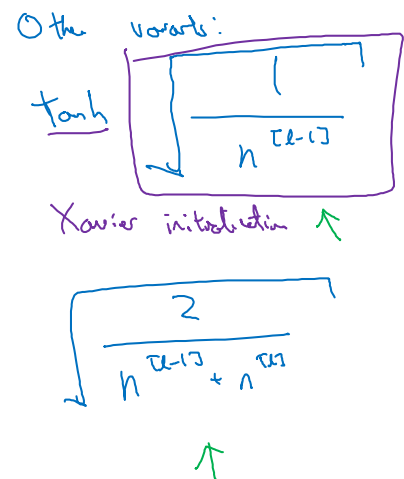
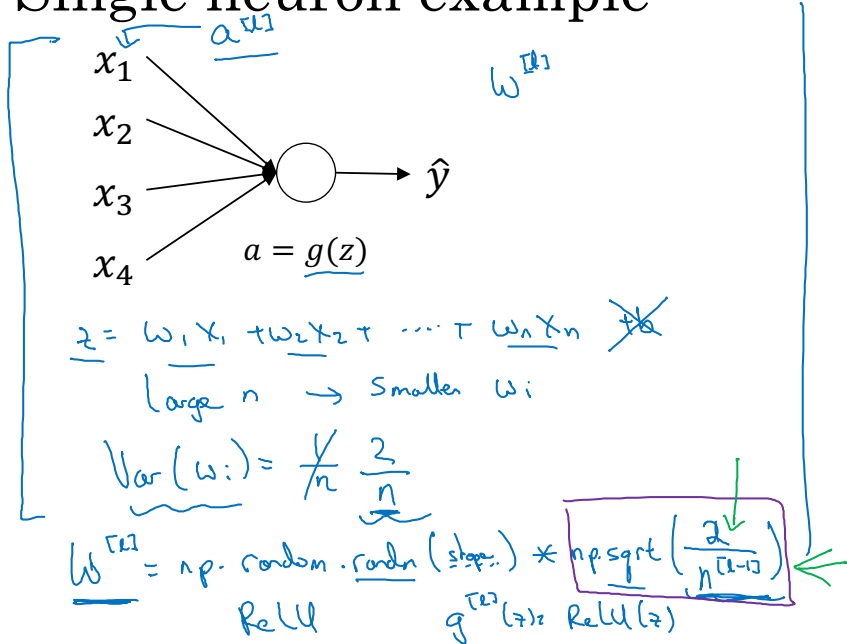
Setting up your optimization problem

Vanishing/exploding gradients

Vanishing/exploding gradients



Single neuron example



Andrew Ng

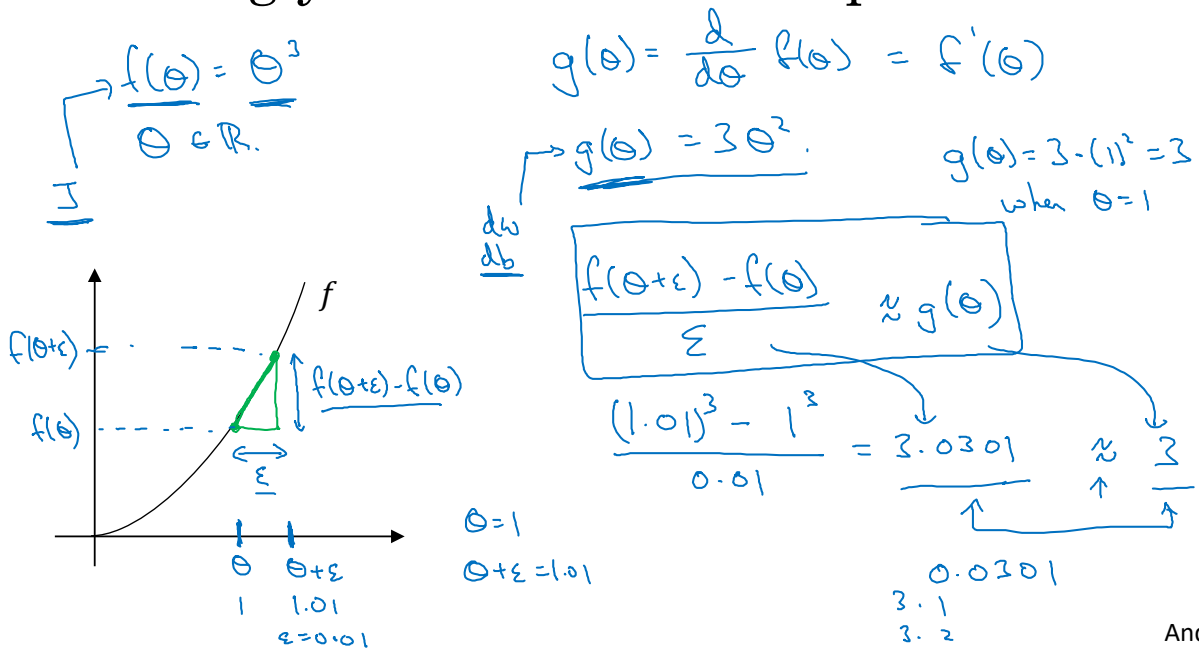


deeplearning.ai

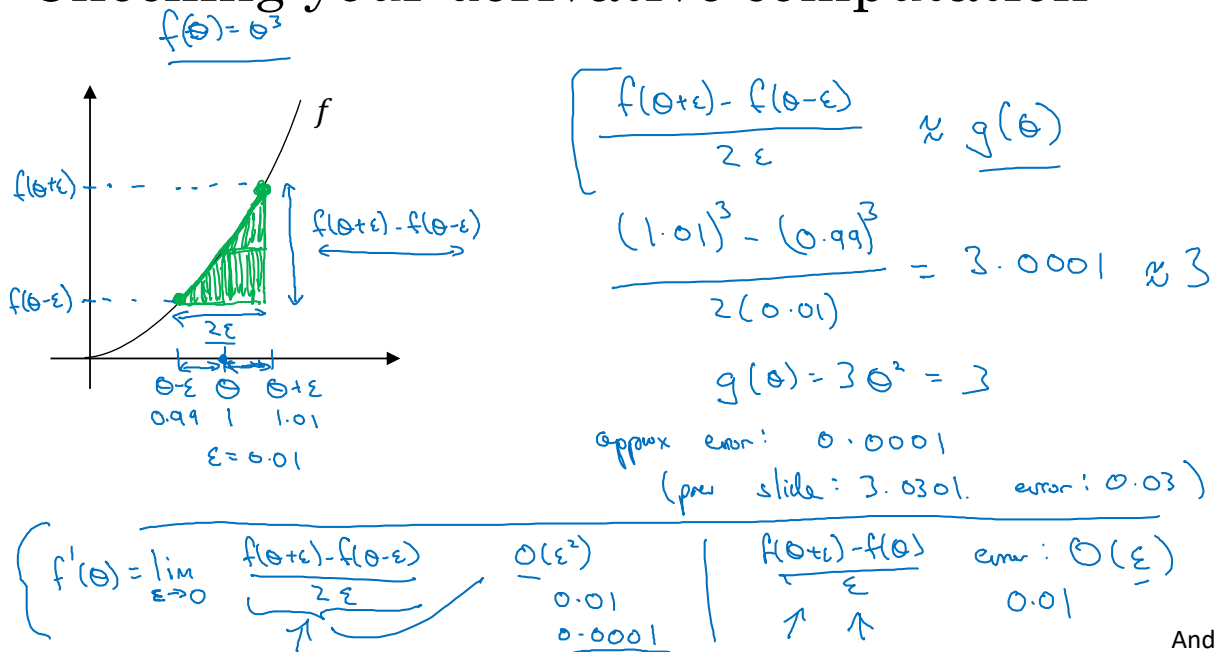
Setting up your optimization problem

Numerical approximation of gradients

Checking your derivative computation



Checking your derivative computation





deeplearning.ai

Setting up your optimization problem

Gradient Checking

Gradient check for a neural network

Take $W^{[1]}, b^{[1]}, \dots, W^{[L]}, b^{[L]}$ and reshape into a big vector θ .

Handwritten note: $J(W^{[1]}, b^{[1]}, \dots, W^{[L]}, b^{[L]}) = J(\theta)$

Take $dW^{[1]}, db^{[1]}, \dots, dW^{[L]}, db^{[L]}$ and reshape into a big vector $d\theta$.

Handwritten note: $J(\theta)$

Handwritten question: Is $d\theta$ the gradient of $J(\theta)$?

Gradient checking (Grad check)

$$J(\theta) = J(\theta_1, \theta_2, \dots)$$

for each i :

$$\rightarrow \underline{d\theta_{\text{approx}}[i]} = \frac{J(\theta_1, \theta_2, \dots, \theta_i + \epsilon, \dots) - J(\theta_1, \theta_2, \dots, \theta_i - \epsilon, \dots)}{2\epsilon}$$

$$\approx \underline{d\theta[i]} = \frac{\partial J}{\partial \theta_i} \quad \Bigg| \quad d\theta_{\text{approx}} \stackrel{?}{\approx} d\theta$$

Check $\frac{\|d\theta_{\text{approx}} - d\theta\|_2}{\|d\theta_{\text{approx}}\|_2 + \|d\theta\|_2}$

$\epsilon = 10^{-7}$

$\approx \frac{10^{-7}}{10^{-5}} - \text{great!} \leftarrow$

$\rightarrow 10^{-3} - \text{worry.} \leftarrow$

Andrew Ng



deeplearning.ai

Setting up your optimization problem

Gradient Checking implementation notes

Gradient checking implementation notes

- Don't use in training – only to debug

$$\frac{d\theta_{approx}[i]}{\uparrow \uparrow} \longleftrightarrow \frac{d\theta[i]}{\uparrow}$$

- If algorithm fails grad check, look at components to try to identify bug.

$$\frac{db^{[L]}}{\uparrow} \quad \frac{dw^{[L]}}{\uparrow}$$

- Remember regularization.

$$\mathcal{J}(\theta) = \frac{1}{n} \sum_i \mathcal{L}(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2n} \sum_l \|w^{[L]}\|_F^2$$

$d\theta = \text{grad of } \mathcal{J} \text{ wrt. } \theta$

- Doesn't work with dropout.

$$\mathcal{J} \quad \text{keep-prob} = 1.0$$

- Run at random initialization; perhaps again after some training.

$$\underline{w, b} \approx 0$$